```python
# ---------------------------------------------------------
# Two-Period Consumption Problem
# ---------------------------------------------------------

import os
import sys

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize, LinearConstraint, Bounds

# Printing the outputs as a text file if ==1
output_file = 0

# ---------------------------------------------------------
# Problem Set-up
# ---------------------------------------------------------
beta = 0.9

def fun(x):
    return -(np.log(x[0]) + beta * np.log(x[1])) # negative

R0 = 1.05
a_rhs = 1.0

A = np.array([1.0, 1.0 / R0])
b = a_rhs

x0 = np.array([0.5, 0.5])

# intertemporal budget constraint: x1 + x2/R0 <= a_rhs
lincon = LinearConstraint([A], -np.inf, b)

# non-negativity: x1 >= 0, x2 >= 0
bounds = Bounds([0.0, 0.0], [np.inf, np.inf])

# ---------------------------------------------------------
# minimize (trust-constr)
# ---------------------------------------------------------
result = minimize(fun, x0, method='trust-constr',
                  constraints=[lincon], bounds=bounds)

x = result.x
fval = result.fun
lam_budget = result.v[0]    # multiplier for the intertemporal budget constraint
lam_bounds = result.v[1]    # multipliers for [x1>=0, x2>=0]

if output_file == 1:
    dfile = "two_periods_consumption_fmincon.txt"
    if os.path.exists(dfile):
        os.remove(dfile)
    sys.stdout = open(dfile, "w")

# ---------------------------------------------------------
# Output
# ---------------------------------------------------------
print(f"beta={beta:.3f}, R0={R0:.3f}, A={a_rhs:.3f}")
```

```
print(f"Optimal consumption in two periods: {x[0]:.3f},{x[1]:.3f}")
print(f"Lifetime Utility: {-fval:.5f}")
print(f"Multiplier (intertemporal budget const): {lam_budget[0]:.5f}")
print(f"Multiplier (non-negativity const): {lam_bounds[0]:.5f}, {lam_bounds[1]:.5f}")

if output_file == 1:
    sys.stdout.close()
    sys.stdout = sys.__stdout__

# ---------------------------------------------------------
# Plot
# ---------------------------------------------------------
x1_vec = np.linspace(0, a_rhs, 100)
x2_vec = np.linspace(0, R0 * a_rhs, 100)
x1_mesh, x2_mesh = np.meshgrid(x1_vec, x2_vec)

# suppress the RuntimeWarning rather than avoid the boundary point due to log(0).
with np.errstate(divide='ignore'):
    utility_mesh = np.log(x1_mesh) + beta * np.log(x2_mesh)

fig, ax = plt.subplots()
ax.grid(True)

ax.plot(x1_vec, R0 * (a_rhs - x1_vec), 'k', linewidth=1) # intertemporal budget constraint
ax.contour(x1_mesh, x2_mesh, utility_mesh, levels=[-fval],
           colors='b', linewidths=1) # indifference curve
neighbor_levels = sorted([0.75 * (-fval), 1.25 * (-fval)]) # 0.75*(-fval) > 1.25*(-fval)
ax.contour(x1_mesh, x2_mesh, utility_mesh,levels=neighbor_levels,
           colors='k', linewidths=1, linestyles='--') # neighboring indifference curves
ax.plot(x[0], x[1], 'ro', linewidth=1) # optimal point
ax.set_xlabel(r'$x_{1}$', fontsize=12)
ax.set_ylabel(r'$x_{2}$', fontsize=12)
ax.set_title('Two-Period Consumption Problem', fontsize=14)
ax.set_xlim([0, a_rhs])
ax.set_ylim([0, R0 * a_rhs])
plt.show()
```